

INTERVIEW PREPARATION & PROJECT HANDBOOK

LuxeHome – Premium Home Services Platform

A comprehensive, end-to-end full-stack web application designed for on-demand luxury home services with Role-Based Access Control, secure bcrypt authentication, and full administrative management.

FRONTEND ARCHITECTURE

React 19 + Vite, Glassmorphism UI design, Dynamic Single Page Application routing, responsive layout, modal dialogs, and real-time state management.

BACKEND & API

Node.js & Express REST API, `mysql2` connection pooling, parameterized query execution, role-based authorization middleware (RBAC), and IP logging.

DATABASE & SECURITY

MySQL Relational Database, `bcryptjs` salted password hashing (10 salt rounds), dual-identifier authentication, and strict SQL injection defenses.

USER ROLES & RBAC

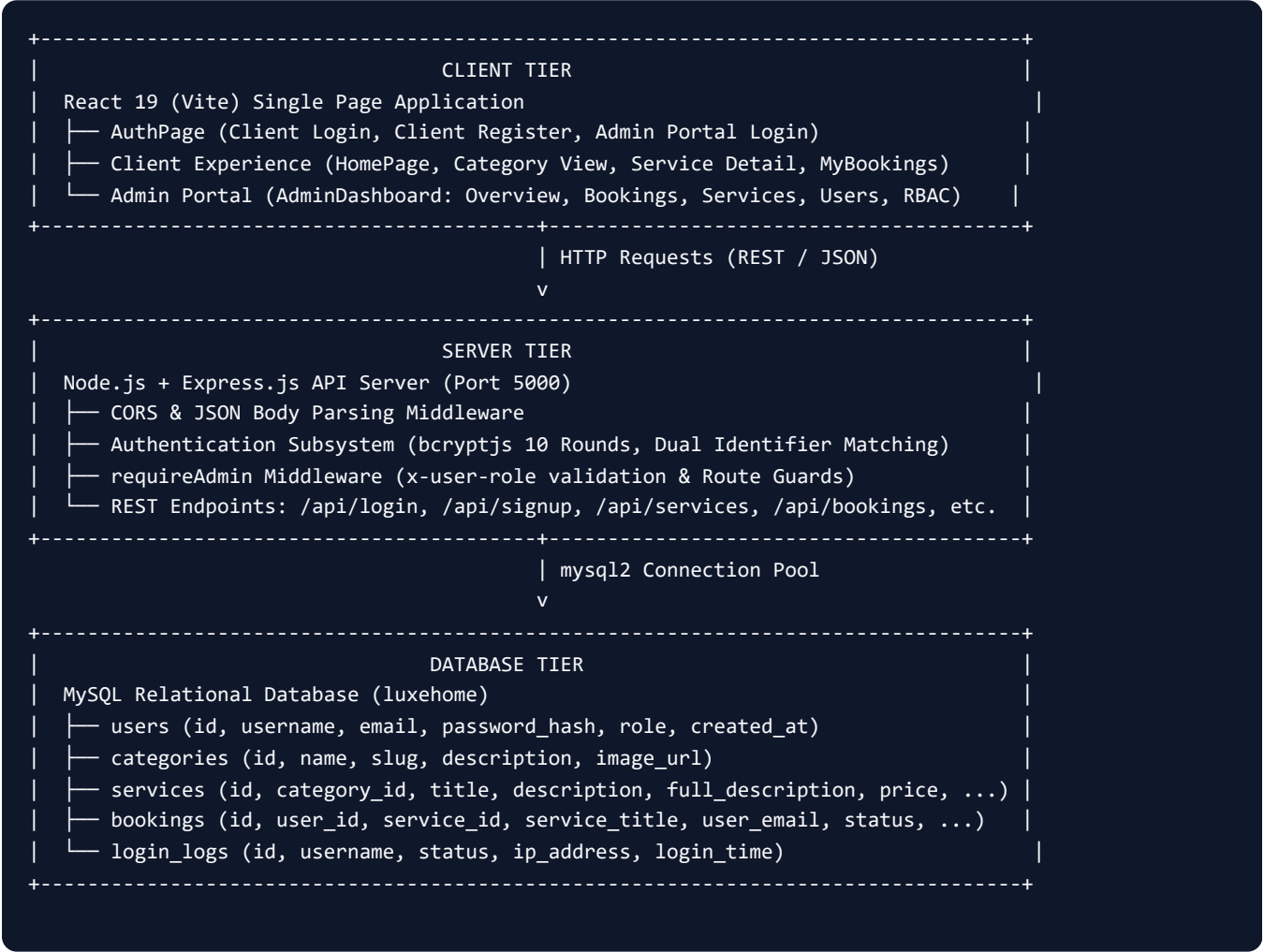
Dedicated Client Experience vs Administrative Access Portal with Route Guards, Bookings approval/rejection, Service Provider assignment, and Analytics.

1. Executive Summary & Architecture Overview

LuxeHome is an enterprise-grade, luxury home service booking and management platform. It bridges high-end residential clients with certified service professionals across multiple domains including Cleaning, Interior Design, Plumbing, Smart Security, Garden Care, and HVAC.

Key Architectural Highlight: The application enforces a strict separation between **Client Services** and **Administrative Operations** at both the database, backend API, and React frontend routing levels.

High-Level Architecture Diagram



2. Authentication & Role-Based Access Control (RBAC)

The authentication system is engineered around two core pillars: **Security** and **Role Isolation**.

Client Role (role = 'client')

- Public self-registration enabled.
- Always assigned `role = 'client'` on signup.
- Can discover services, view details, and book services.
- Isolated access: Can only view and cancel their own bookings (`/api/bookings/user/:userId`).
- Forbidden from administrative routes and mutations.

Admin Role (role = 'admin')

- No public admin self-registration (prevents unauthorized privilege escalation).
- Pre-seeded secure account in database with bcrypt hash.
- Accesses system through dedicated **Admin Login** portal.
- Full CRUD on Services & Categories.
- Full control over Bookings (Approve, Reject, In Progress, Complete).
- User administration (Promote, Demote, Delete users).

1. Secure Password Hashing (bcryptjs)

All passwords are encrypted with bcrypt using a workload factor of **10 salt rounds** before any database insertion. Plaintext passwords are never stored.

```
// Registration Endpoint (backend/server.js)
app.post('/api/signup', async (req, res) => {
  const { username, email, password } = req.body;
  const trimmedUsername = username.trim();
  const trimmedEmail = email.trim().toLowerCase();
  const trimmedPassword = password.trim();

  // Secure bcrypt hashing
  const saltRounds = 10;
  const hashedPassword = await bcrypt.hash(trimmedPassword, saltRounds);

  // Hardcoded 'client' role protects against privilege escalation
  const sql = 'INSERT INTO users (username, email, password_hash, role) VALUES (?, ?, ?, \'client\')';
  db.query(sql, [trimmedUsername, trimmedEmail, hashedPassword], (err, results) => {
    if (err && err.code === 'ER_DUP_ENTRY') {
      return res.status(400).json({ success: false, message: 'Username or Email already exists' });
    }
    res.json({ success: true, message: 'User registered successfully', userId: results.insertId });
  });
});
```

2. Dual Identifier Login & Case-Insensitive Credential Verification

Users can seamlessly authenticate using **either their registered Email Address OR their Username**. The lookup executes a case-insensitive match against MySQL followed by asynchronous `bcrypt.compare()`.

```
// Login Endpoint (backend/server.js)
app.post('/api/login', (req, res) => {
  const { username, email, password } = req.body;
  const identifier = (username || email || '').trim();
  const inputPassword = (password || '').trim();

  // Dual-identifier lookup query
  const sql = 'SELECT * FROM users WHERE LOWER(email) = LOWER(?) OR LOWER(username) = LOWER(?) LIMIT 1';
  db.query(sql, [identifier, identifier], async (err, results) => {
    if (err) return res.status(500).json({ error: err.message });
    const user = results[0];

    if (user && await bcrypt.compare(inputPassword, user.password_hash)) {
      return res.json({
        success: true,
        user: { id: user.id, username: user.username, email: user.email, role: user.role }
      });
    }
    res.status(401).json({ success: false, message: 'Invalid username/email or password' });
  });
});
```

3. Backend Authorization Guard (requireAdmin)

```
const requireAdmin = (req, res, next) => {
  const userRole = req.headers['x-user-role'];
  if (userRole === 'admin') {
    next();
  } else {
    res.status(403).json({ success: false, message: 'Access denied. Admins only.' });
  }
};
```

3. Relational Database Design & Schema

The MySQL relational database `luxehome` is normalized with foreign keys and cascading delete constraints.

Table Name	Key Columns & Types	Constraints / Indexes	Purpose
users	id (INT PK AI) username (VARCHAR 255) email (VARCHAR 255) password_hash (VARCHAR 255) role (VARCHAR 50)	UNIQUE(username) UNIQUE(email) DEFAULT role='client'	Stores authenticated accounts, roles, and bcrypt password hashes.
categories	id (INT PK AI) name (VARCHAR 255) slug (VARCHAR 255) description (TEXT) image_url (VARCHAR 511)	UNIQUE(slug)	Groups services into domains (Cleaning, HVAC, Plumbing, Interior, Security, Garden).
services	id (INT PK AI) category_id (INT FK) title (VARCHAR 255) price (VARCHAR 50) duration (VARCHAR 100) image_url (VARCHAR 511)	FOREIGN KEY -> categories(id) ON DELETE CASCADE	Catalog of luxury services with rich descriptions, pricing, and timing.
bookings	id (INT PK AI) user_id (INT) service_id (INT FK) service_title (VARCHAR 255) user_email (VARCHAR 255)	FOREIGN KEY -> services(id) ON DELETE SET NULL DEFAULT status='Pending'	Tracks customer bookings and lifecycle state (Pending, Confirmed, Completed, Cancelled).

Table Name	Key Columns & Types	Constraints / Indexes	Purpose
	status (VARCHAR 50)		
login_logs	id (INT PK AI) username (VARCHAR 255) status (VARCHAR 50) ip_address (VARCHAR 45) login_time (TIMESTAMP)	Audit Log	Security audit log recording all login attempts, success/failure status, and client IP.

4. Complete REST API Reference

Method	Endpoint	Access Level	Description
POST	/api/signup	Public	Registers a new client with bcrypt password hashing.
POST	/api/login	Public	Authenticates client or admin using email/username + password.
GET	/api/categories	Public / Client	Retrieves all service categories.
GET	/api/services	Public / Client	Retrieves all catalog services with category names.
GET	/api/services/:slug	Public / Client	Retrieves services filtered by category slug.
POST	/api/bookings	Client	Creates a new service booking for a user.
GET	/api/bookings/user/:userId	Client / Admin	Fetches all bookings specific to a single user.
GET	/api/bookings	Admin Only	Fetches all bookings across all clients for management.
PUT	/api/bookings/:id	Admin Only	Updates booking status (Pending, Confirmed, Completed, Cancelled).
POST	/api/services	Admin Only	Creates a new service item in the catalog.
PUT	/api/services/:id	Admin Only	Edits service title, description, price, duration, image.
DELETE	/api/services/:id	Admin Only	Deletes a service item permanently.
GET	/api/users	Admin Only	Retrieves all registered users with role information.
PUT	/api/users/:id/role	Admin Only	Promotes or demotes user role between client and admin.
DELETE	/api/users/:id	Admin Only	Deletes a user account permanently.

5. Top Technical Interview Questions & Answers

Q1 Can you explain the overall architecture and tech stack of LuxeHome?

Answer: LuxeHome is built on a 3-tier architecture:

- **Frontend:** React 19 bootstrapped with Vite. It features a custom luxury glassmorphic theme in vanilla CSS, responsive modular components, and state-driven route management with route protection guards.
- **Backend:** A lightweight REST API built with Node.js and Express.js, featuring CORS, middleware-based RBAC authorization, and IP-auditing login logs.
- **Database:** MySQL with connection pooling via `mysql2`, featuring normalized tables, foreign key constraints with cascade deletes, and unique constraint validations.
- **Security:** Password hashing using `bcryptjs` with 10 salt rounds, parameterized SQL queries preventing SQL injection, and strict role isolation between clients and administrators.

Q2 What authentication bug did you encounter and how did you resolve it?

Answer:

Problem: When users registered with an email, username, and password, subsequent logins with their email address failed with 401 Unauthorized. The root cause was that the backend SQL query strictly queried `WHERE username = ? AND password_hash = ?`, and passwords were being compared in plain text.

Solution:

1. Integrated `bcryptjs` to hash passwords during registration using `bcrypt.hash(password, 10)`.
2. Refactored `/api/login` to perform a dual-identifier lookup: `SELECT * FROM users WHERE LOWER(email) = LOWER(?) OR LOWER(username) = LOWER(?) LIMIT 1`.
3. Used `await bcrypt.compare(inputPassword, user.password_hash)` to validate credentials safely and added automatic legacy password migration.

Q3 How is Role-Based Access Control (RBAC) enforced across the stack?

Answer: RBAC is enforced at 3 distinct layers:

- **Registration Level:** `/api/signup` hardcodes `role = 'client'`, preventing any user from injecting an `admin` role during public signups.
- **Backend Middleware:** Administrative endpoints are guarded by `requireAdmin` middleware which validates the user's role header and returns `403 Forbidden` for clients.
- **Frontend Route Guard:** In `App.jsx`, if an unauthenticated user or client attempts to load the Admin Dashboard, state-based route guards immediately redirect them to Home.

Q4 How do you prevent SQL Injection and protect database integrity?

Answer: All database interactions utilize **parameterized queries** (e.g. `db.query('SELECT * FROM users WHERE email = ?', [email])`). This ensures user inputs are treated strictly as data literals and never interpreted as executable SQL code. Furthermore, relational integrity is maintained using Foreign Keys (e.g. cascading deletes on services when a category is removed, and `ON DELETE SET NULL` for bookings when a service is removed).

Q5 How would you scale this application for enterprise production?

Answer:

1. **Authentication:** Implement JWT (JSON Web Tokens) with short-lived access tokens and HTTP-only secure refresh tokens stored in cookies.
2. **Caching:** Introduce Redis to cache static service catalogs and category queries to reduce database read load.
3. **Payments:** Integrate Stripe or PayPal webhooks for end-to-end automated payment capture and refund processing.
4. **Real-Time Updates:** Use WebSockets (Socket.io) for real-time booking status notifications to clients and admins.

6. Quick Demo & Credentials Cheatsheet

Administrator Credentials

Username: admin

Email: admin@gmail.com

Password: 1234

Login Path: Select *Admin Login* tab on Auth screen.

Dashboard Capabilities: KPI metrics overview, real-time booking status changes, service creation/editing, and user management.

Client Demo Credentials

Username: testuser

Email: testuser@gmail.com

Password: password123

Or Register: Click *Register* to create any new client account.

Client Capabilities: Browse services by category, book appointments, track personal bookings in *My Bookings*.

LuxeHome Project Master Reference Guide • Prepared for Full-Stack / Software Engineering Interviews